Mathematical modelling of T-Adaptive Segment Routing

Project Orange – Challenge EURO/ROADEF 2026

Nguyễn Tiến Đức (20252076M) Hoàng Huy Chiến (20261069M)

IT4033E Capstone, SoICT

October 2026

Good morning. We are Nguyen Tien Duc and Hoang Huy Chien. Our capstone project is Project Orange, taken from the EURO/ROADEF 2026 challenge: T-Adaptive Segment Routing. In about ten minutes we will describe the problem, our MILP model, how we validated it, and what happened when we solved the twenty instances of set A with an open-source solver.

Problem: T-Adaptive Segment Routing

- IP backbone $G = (V, A)$, link metric $\omega(a)$, capacity $c(a)$; demands $d = (s_d, t_d)$ with volume $\nu(d, t)$.
- Packets follow shortest paths; ties are split (ECMP). Congestion = high load $\lambda(a, t) = \text{flow}(a, t)/c(a)$.
- **Segment Routing**: each demand gets a list of at most $\mathit{maxSeg} - 1$ *waypoints*; shortest-path routing between consecutive waypoints $\Rightarrow$ bend flows away from hot links.
- Periods $t = 0$ (nominal) and $t = 1$ (links $q(1)$ down); between periods at most $\kappa(1)$ segment changes.

Goal

Choose a segment path per demand and period to minimise link loads. Experiments: period 0 alone, then periods $\{0, 1\}$ with budget, on the 20 instances of set A.

The setting is an IP backbone network. Each link has a routing metric and a capacity, and each demand is a source-target pair with a traffic volume. Routers send packets along shortest paths and split ties evenly, which is called ECMP. Congestion is measured by link utilisation: flow divided by capacity.

Segment Routing lets the operator add up to maxSeg minus one waypoints to a demand. Between two consecutive waypoints the traffic still follows shortest paths, so waypoints bend flows away from hot links without changing the metrics.

The T-adaptive part: there are several periods. Period 0 is the nominal network; in period 1 some links are down. Between periods we may change at most kappa segments, so we cannot re-optimise everything from scratch.

Our goal is to choose one segment path per demand and per period that minimises link load.

We ran two experiments: period 0 alone, then periods 0 and 1 together with the change budget.

Data and split coefficients $r(i, j, a, t)$

- Input: `net.json` (ω, c), `tm.json` (ν), `scenario.json` ($maxSeg$, κ, q).
- $r(i, j, a, t)$ = fraction of a unit flow $i \to j$ in $G(t)$ that crosses a (course document):

$$r(i, j, a, t) = \frac{\sigma_{iu}(t)\,\sigma_{vj}(t)}{\sigma_{ij}(t)} \quad \text{if } d_{iu} + \omega(a) + d_{vj} = d_{ij},\ a = (u, v),\ \text{else } 0$$

  σ = number of shortest paths: one Dijkstra with path counting per source, $O(nm \log n)$.
- Path counting $\neq$ per-node ECMP: toy instance $r(v_0, v_5, a_{1,3}, 0) = 1/3$ vs. $1/2$ in the subject's table. We keep the course definition.

The input comes in three JSON files: the network, the traffic matrix, and the scenario with maxSeg, the budget kappa and the failed links.
The key precomputed data is r(i,j,a,t): the fraction of one unit of flow from i to j that crosses link a in period t. Following the course document, it is sigma(i,u) times sigma(v,j) over sigma(i,j), where sigma counts shortest paths, and it is nonzero only if a lies on a shortest path. We compute it with one Dijkstra with path counting per source.
One point we want to be transparent about: path counting is not the same as per-node ECMP splitting. On the toy instance we get one third where the subject's table says one half. We kept the course definition and documented the difference.

MILP model

Variables

- $x_{ij}^{dt} \in \{0,1\}$: segment $(i,j) \in K$ used by demand d at period t (K = reachable pairs).
- $U \geq 0$: maximum link utilisation (MLU); $y_{ij}^{dt} \geq 0$ linearises $|x_{ij}^{dt} - x_{ij}^{d,t-1}|$.

Objective min U (first criterion of the subject's lexicographic objective).

$\sum_j x_{ij}^{dt} - \sum_j x_{ji}^{dt} = \mathbb{1}[i = s_d] - \mathbb{1}[i = t_d]$	(1) path in the segment graph
$\sum_{(i,j)} x_{ij}^{dt} \leq maxSeg$	(2) at most 5 waypoints
$\sum_d \sum_{(i,j)} r(i,j,a,t)\, \nu(d,t)\, x_{ij}^{dt} \leq U\, c(a) \quad \forall a \notin q(t)$	(3) load
$\sum_{d,(i,j)} y_{ij}^{dt} \leq \kappa(t), \quad y \geq x^t - x^{t-1}, \; y \geq x^{t-1} - x^t$	(4) change budget

Size: $|D|\,|K|\,|T|$ binaries, $|K| \leq n(n-1) \Rightarrow$ from $1.5 \cdot 10^4$ (inst. 01) to $\approx 10^9$ (inst. 20).

The variables: x is one if demand d uses segment (i,j) in period t; K is the set of reachable pairs. U is the maximum link utilisation, and y linearises the absolute change between consecutive periods.
We minimise U, the first criterion of the subject's lexicographic objective.
Constraint 1 is flow conservation in the segment graph, so the chosen segments form a path from source to target. Constraint 2 limits the number of segments. Constraint 3 says the load on each working link, built from the r coefficients and the volumes, is at most U times the capacity. Constraint 4 is the change budget between periods.
The weak point is size: binaries = demands times segments times periods. It goes from fifteen thousand on instance 1 to about one billion on instance 20. This drives everything you will see in the results.

Implementation and validation

- Julia 1.13 + JuMP 1.32 + HiGHS 1.15 (open-source branch-and-bound).
- **Validation first**: 24 automated tests on the subject's toy instance; MILP optimum = brute-force enumeration of all segment paths (MLU 0.375, waypoint v_4).
- **MIP start**: no-waypoint routing ($x_{s_d t_d} = 1$) is always feasible with 0 changes; its MLU is the reference column *no-wp*.
- **Protocol**: 10 runs per instance (HiGHS seeds 1–10, 1 thread each), time limit 1800 s (period 0) / 3600 s (periods 0+1); gap $= 1 - \mathrm{LB}/\mathrm{UB}$; CPU = `optimize!` time only.
- **Machine**: i9-14900K, 125 GB RAM (shared server), runs in parallel through a memory-aware job queue.

We implemented the model in Julia with JuMP and the open-source solver HiGHS.
Before any experiment we validated it on the subject's toy instance: 24 automated tests, and the MILP optimum equals brute-force enumeration of all segment paths, MLU 0.375 with waypoint v4.
We give HiGHS a MIP start: routing with no waypoints is always feasible and uses zero changes. Its MLU is the reference column no-wp in the tables.
Protocol: ten runs per instance, seeds 1 to 10, one thread each, time limit 30 minutes for period 0 and one hour for two periods. Gap is one minus lower bound over upper bound; CPU time is the optimize call only.
Everything ran on a shared server, i9-14900K with 125 GB of RAM, through a job queue that only starts a job when its measured memory fits.

Results – period 0 (averages over 10 runs)

Inst	n	m	$\lvert D\rvert$	vars	opt/runs	MLU	no-wp	gap (%)	CPU (s)	nodes
01	20	80	40	0.02M	10/10	0.929	1.000	0.0	4.0	1.0
02	30	150	45	0.04M	10/10	0.549	1.000	0.0	27.8	1.0
03	50	250	20	0.05M	10/10	0.944	1.000	0.0	16.3	1.0
04	50	250	200	0.49M	0/10	0.675	0.692	31.5	1819.3	0.0
05	100	396	100	0.99M	4/10	0.135	0.333	12.6	1578.5	0.4
06	100	500	500	4.95M	0/10	1.000	1.000	100.0	2056.4	0.0
07	100	500	800	7.92M	2/10	0.908	0.908	80.0	2069.4	0.2
08	150	654	200	4.47M	1/10	0.800	1.000	53.8	12450.4	0.1
09	150	750	200	4.47M	0/10	0.611	0.611	17.3	2403.3	0.0
10	150	966	1000	22.35M	0/10	1.000	1.000	100.0	2989.6	0.0
11	200	1000	400	15.92M	0/10	1.000	1.000	100.0	2430.3	0.0
12	200	898	400	15.92M	0/10	1.000	1.000	100.0	2342.9	0.0
13	200	1000	2000	79.60M	model too large					
14	250	1108	600	37.35M	0/10	1.000	1.000	100.0	3032.5	0.0
15	250	1250	600	37.35M	0/10	1.000	1.000	100.0	2826.6	0.0
16	250	1452	4800	298.80M	model too large					
17	300	1270	2000	179.40M	model too large					
18	300	1500	2000	179.40M	model too large					
19	300	1998	6000	538.20M	model too large					
20	400	2000	6000	957.60M	model too large					

Period 0, averaged over ten runs. Columns: instance size, number of proved optima, average and best MLU, best bound, gap, time, branch-and-bound nodes, and the no-waypoint MLU for reference.

Read it top to bottom: small instances are solved, medium ones partly, large ones only return the MIP start, and the last ones are marked too large to build. The next slide explains each group.

Period 0 – observations

- **01–03**: optimal at the root node in seconds; waypoints lower the MLU from 1.00 to 0.93, 0.55, 0.94.
- **04–05**: no proof within 30 min, time spent in the root node (0 B&B nodes). Strong seed effect: on 05, 4/10 seeds prove the optimum 0.1155; on 04 only 1/10 improves the MIP start. $\Rightarrow$ averaging over 10 seeds is necessary.
- **06–12** (millions of binaries): the LP relaxation takes most of the time; usually the MIP start is returned. 08: one seed proves 0.319, 3 stop at 0.561, 6 keep 1.000. 10–12, 14, 15: all runs stop with the MIP start and no lower bound (gap 100%).
- HiGHS does not always respect the time limit: up to 51940 s for a 1800 s limit (instance 08).
- **13–20**: up to $9.6 \cdot 10^8$ binaries; $\approx$ 2 GB per million variables $\Rightarrow$ only 14, 15 fit in 125 GB.

Instances 1 to 3 are solved to optimality at the root node in seconds. Waypoints reduce the MLU from 1.00 to 0.93, 0.55 and 0.94; instance 2 shows the strongest gain.
Instances 4 and 5 are not proved within 30 minutes; all the time is spent in the root node. The seed matters a lot: on instance 5, four of ten seeds prove the optimum 0.1155; on instance 4 only one seed improves the MIP start. That is why we average over ten seeds.
From instance 6 on there are millions of binaries. The LP relaxation takes most of the time and usually the MIP start is returned. On instance 8 one seed proves 0.319, three stop at 0.561 and six keep 1.0. Instances 10 to 12, 14 and 15 all stop with the MIP start and no lower bound.
HiGHS does not always respect the time limit: one run on instance 8 took 51,940 seconds for an 1,800-second limit. We report measured times as they are.
Instances 13 and 16 to 20 need up to 960 million binaries. At about 2 GB per million variables, only 14 and 15 fit in 125 GB.

Results – periods 0 and 1 (budget, presolve off)

Inst	n	m	$\lvert D\rvert$	vars	opt/runs	MLU	no-wp	gap (%)	CPU (s)	nodes
01	20	80	40	0.05M	10/10	0.929	1.000	0.0	23.4	11.0
02	30	150	45	0.12M	7/10	0.942	1.098	5.2	2386.5	6.1
03	50	250	20	0.15M	3/10	0.961	1.000	24.0	3109.1	44.4
04	50	250	200	1.47M	0/10	0.898	1.000	32.9	3638.6	0.0
05	100	396	100	2.97M	0/10	0.865	1.000	77.1	5198.7	0.0
06	100	500	500	14.85M	0/5	1.000	1.000	100.0	3893.4	0.0
08	150	654	200	13.41M	0/1	1.000	1.000	68.1	3892.2	0.0
09	150	750	200	13.41M	0/5	1.000	1.000	66.0	3869.9	0.0
10	150	966	1000	67.05M	model too large					
11	200	1000	400	47.76M	model too large					
12	200	898	400	47.76M	model too large					
13	200	1000	2000	238.80M	model too large					
14	250	1108	600	112.05M	model too large					
15	250	1250	600	112.05M	model too large					
16	250	1452	4800	896.40M	model too large					
17	300	1270	2000	538.20M	model too large					
18	300	1500	2000	538.20M	model too large					
19	300	1998	6000	1614.60M	model too large					
20	400	2000	6000	2872.80M	model too large					

The two-period experiment with the change budget, run with presolve off; the next slide explains why. Rows with fewer than ten runs, and rows marked too large, come from time and memory limits, which the report states explicitly.

Two periods – observations

- Variables $\times 3$ (two copies of x plus y).
- **Presolve stall**: with HiGHS defaults, instance 04 stayed in presolve $>$ 3 h for a 1 h limit. Ablation (60 s limit): removing (4) *or* presolve off $\Rightarrow$ limit respected. $\Rightarrow$ the dense budget row blocks presolve; all two-period runs use presolve off.
- Instance 02: link down makes the no-waypoint MLU 1.098 (congestion); optimum 0.903 is proved by 7/10 seeds with presolve off vs. 2/10 with presolve on.
- Instance 05: 2/10 seeds find 0.323 (no-wp 1.000); the others keep the MIP start.
- Without presolve memory grows to 3.7–5.5 GB per million variables $\Rightarrow$ two-period model of 07 ($\approx$ 88 GB) only alone, 10–20 cannot be built.

With two periods the model has about three times more variables: two copies of x plus y.
First problem: a presolve stall. With default settings, instance 4 stayed in presolve more than three hours with a one-hour limit. A small ablation with a 60-second limit showed that removing the budget constraint, or turning presolve off, each made the limit respected. So the dense budget row blocks presolve, and all two-period runs use presolve off.
Instance 2 is the interesting case: the link failure makes the no-waypoint MLU 1.098, so the network is congested. The optimum 0.903 is proved by seven of ten seeds with presolve off, against two of ten with presolve on.
On instance 5, two seeds find 0.323 against 1.0 without waypoints; the others keep the MIP start.
The price of presolve off is memory: 3.7 to 5.5 GB per million variables. The two-period model of instance 7 needs about 88 GB, so it can only run alone, and instances 10 to 20 cannot be built.

Experimental process: trial and error

Symptom	Diagnosis	Fix / consequence
Toy: $r = 1/3$, subject says $1/2$	path counting $\neq$ per-node ECMP	keep course definition
04: no feasible solution in 300 s	stuck in root node	no-waypoint MIP start
Laptop 24 GB too small	measured $\approx$ 2–3 GB / M vars	125 GB server + memory-aware queue
Two-period build 5428 s	`sum` over JuMP array is quadratic	generator form: 1.4 s
`optimize!` $>$ 3 h, 1 h limit	ablation: presolve $\times$ budget row	presolve off for two periods
Runs far beyond time limit	HiGHS checks limit between steps (likely)	report measured time; reorder queue

Lessons: validate on a toy instance against brute force first; solver settings are part of the protocol; for exact MILPs of this size, *memory* is the first limit.

Since the grading is on scientific method, we logged every problem. The main ones:
The toy split ratio disagreed with the subject: path counting versus ECMP; we kept the course definition.
Instance 4 found no feasible solution in 300 seconds, stuck in the root node: we added the no-waypoint MIP start.
The laptop's 24 GB was too small: we measured memory per million variables, moved to the 125 GB server and wrote a memory-aware queue.
Building the two-period model took 5,428 seconds because sum over a JuMP array is quadratic; the generator form takes 1.4 seconds.
Then the presolve stall, and the runs far beyond the time limit, which we report as measured.
Lessons: validate on a toy instance against brute force first; solver settings are part of the protocol; and for exact MILPs of this size, memory is the first limit, before time.

Conclusion

- Compact MILP for T-ASR (segment variables, path-counting split ratios), validated against brute force.
- Exact solving works for small instances and shows large MLU reductions (02: $1.00 \to 0.55$).
- Beyond $\approx 10^6$ binaries the solver rarely improves the MIP start in the time limit; beyond $\approx 6 \cdot 10^7$ (period 0) the model cannot even be built in 125 GB.
- Next steps: restrict candidate waypoints, column generation on segment paths, full lexicographic objective.

Thank you – questions?

To conclude: we built a compact MILP for T-adaptive segment routing, with segment variables and path-counting split ratios, validated against brute force.
Exact solving works for small instances and gives large MLU reductions, for example 1.00 to 0.55 on instance 2.
Beyond about one million binaries the solver rarely improves the MIP start within the time limit; beyond about 60 million variables in period 0 the model cannot even be built in 125 GB.
Next steps: restrict the candidate waypoints, column generation on segment paths, and the full lexicographic objective.
Thank you; we are happy to take questions.

Backup – periods 0 and 1 with presolve on

Inst	n	m	$\lvert D\rvert$	vars	opt/runs	MLU	no-wp	gap (%)	CPU (s)	nodes
01	20	80	40	0.05M	10/10	0.929	1.000	0.0	32.6	1.0
02	30	150	45	0.12M	2/10	1.053	1.098	42.9	3224.8	0.7
03	50	250	20	0.15M	4/10	0.946	1.000	11.5	2840.8	0.8

Backup, only for questions: two-period results with HiGHS presolve on, small instances. Compare instance 2: two of ten optimal with presolve on versus seven of ten with presolve off.